

Flerdimensionella vektorer

Även i C++ kan man använda sig av *flerdimensionella vektorer*. Det är inget speciellt med dessa om man använt dylika i andra programmeringsspråk. Man använder dock sällan vektorer med fler än två dimensioner. Man deklarerar en flerdimensionell vektor allmänt på följande sätt:

```
datatyp variabelnamn [storlek1][storlek2]...[storlekN];
```

Exempel på en tvådimensionell vektor (matris) som fungerar som ett schackbräde:

```
int Chessboard[8][8];
```

Indexering av flerdimensionella vektorer fungerar som för endimensionella vektorer, med den skillnaden att man skall ha ett index per dimension. Ett exempel på hur en tvådimensionell vektor kan fyllas med data och skrivas ut till skärmen:

```
#include <iostream>
#include <iomanip>

int main () {
    int Tal [8][4];

    // fyll matrisen med data
    for ( int Y = 0; Y < 8; Y++ ) {
        for ( int X = 0; X < 4; X++ ) {
            Tal [Y][X] = Y * 4 + X;
        }
    }

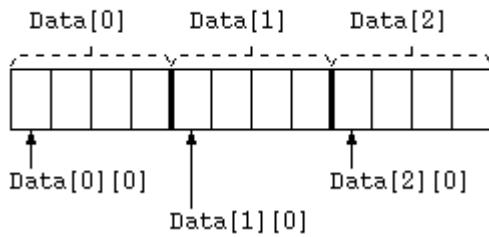
    // skriv ut matrisen till skärmen
    for ( int Y = 0; Y < 8; Y++ ) {
        for ( int X = 0; X < 4; X++ ) {
            cout << setw (3) << Tal [Y][X] << " ";
        }
        cout << endl;
    }
}
```

Flerdimensionella vektorer och minne

I minnet organiseras flerdimensionella vektorer sekventiellt precis som normala endimensionella vektorer. Om man t.ex. har en tvådimensionell vektor kan man se på den som en endimensionell vektor vars element är endimensionella vektorer. Följande figur illustrerar hur vektorns element är upplagda i minnet:

Figur 9-4. Schematisk bild över pekare

```
Data[3][4];
```



Vi ser att man i minnet kan se på `Data` som en serie på tre vektorer med fyra element.

Precis som med vanliga vektorer är flerdimensionella vektorer egentligen pekare till det första elementet. Man kan således även accessera flerdimensionella vektorer med hjälp av normal pekararitmetik (se [avsnittet Pekararitmetik](#)).

[Förutgående](#)

Vektorer som pekare

[Hem](#)

[Upp](#)

[Nästa](#)

Sammansatta datatyper